



Design Architecturte

Ha Noi Aug - 2023

OPEN MIND - OPEN DOORS



Table of content

MVVM Architecture	3
View model	7
Data binding	16
Live data	24
Repository pattern	29

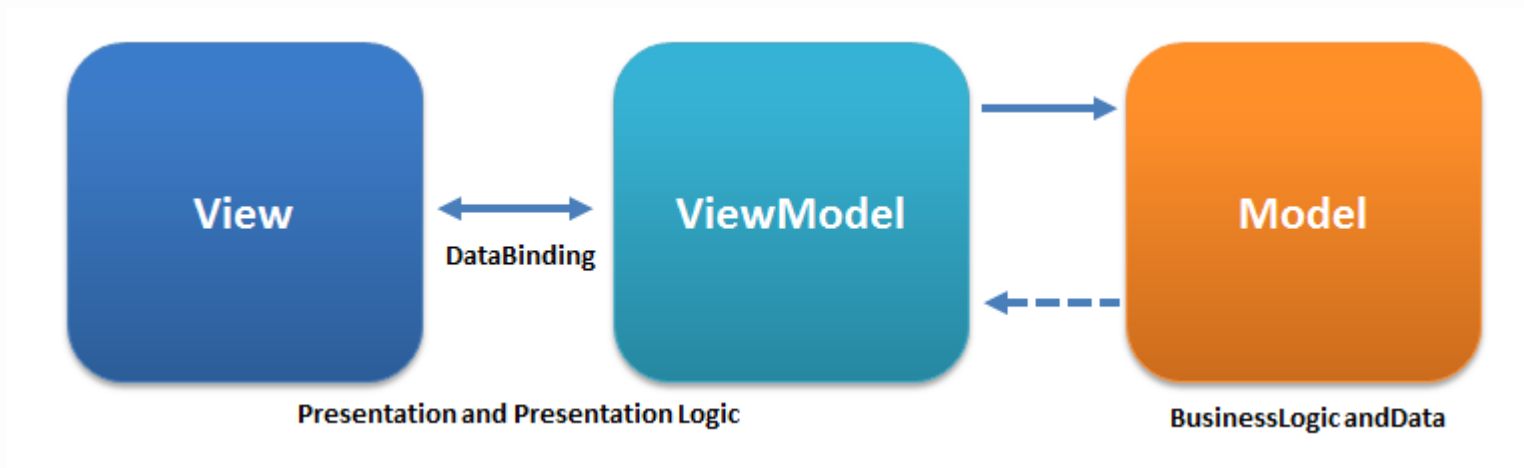
OPEN MIND - OPEN DOORS

MVVM Architecture

OPEN MIND - OPEN DOORS

What is MVVM?

Model-View-ViewModel (MVVM) is a software [design pattern](#) that helps cleanly separate an application's business and presentation logic from its user interface (UI). Maintaining a clean separation between application logic and the UI helps address numerous development issues and makes an application easier to test, maintain, and evolve. It can also significantly improve code reuse opportunities and allows developers and UI designers to collaborate more easily when developing their respective parts of an app.



MVVM

View

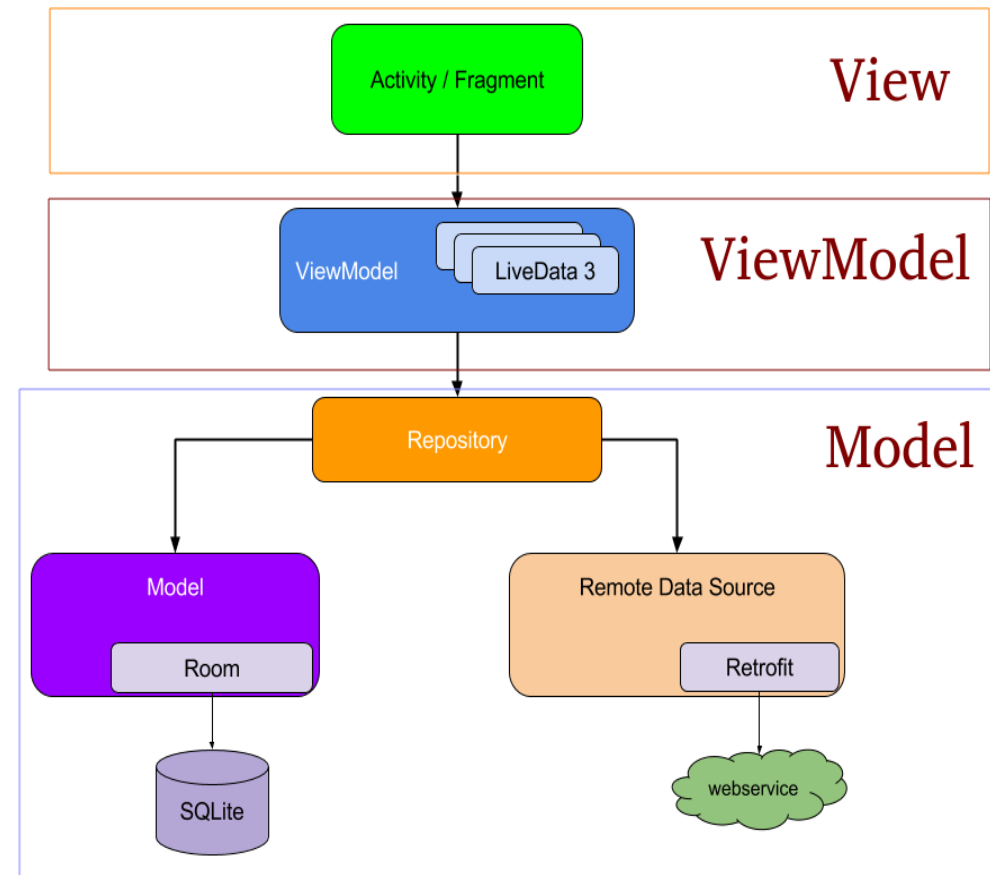
The view is the structure, layout, and appearance of what a user sees on the screen. It displays a representation of the model and receives the user's interaction with the view (mouse clicks, keyboard input, screen tap gestures, etc.), and it forwards the handling of these to the view model via the data binding (properties, event callbacks, etc.) that is defined to link the view and view model.

View Model

The ViewModel is the link between the View and the Model. It implements and exposes public properties and commands that the View uses by way of data binding. If any state changes occur, the ViewModel notifies the View through notification events.

Model

The model represents the app's domain model, which can include a data model as well as business and validation logic. It communicates with the ViewModel, and lacks awareness of the View.



MVVM

ViewModel does not have a reference to a View

View subscribes to streams of data

ViewModel sends data streams to any and all subscribers

ViewModel updates view when data changes

ViewModel interacts with the Model to retrieve data

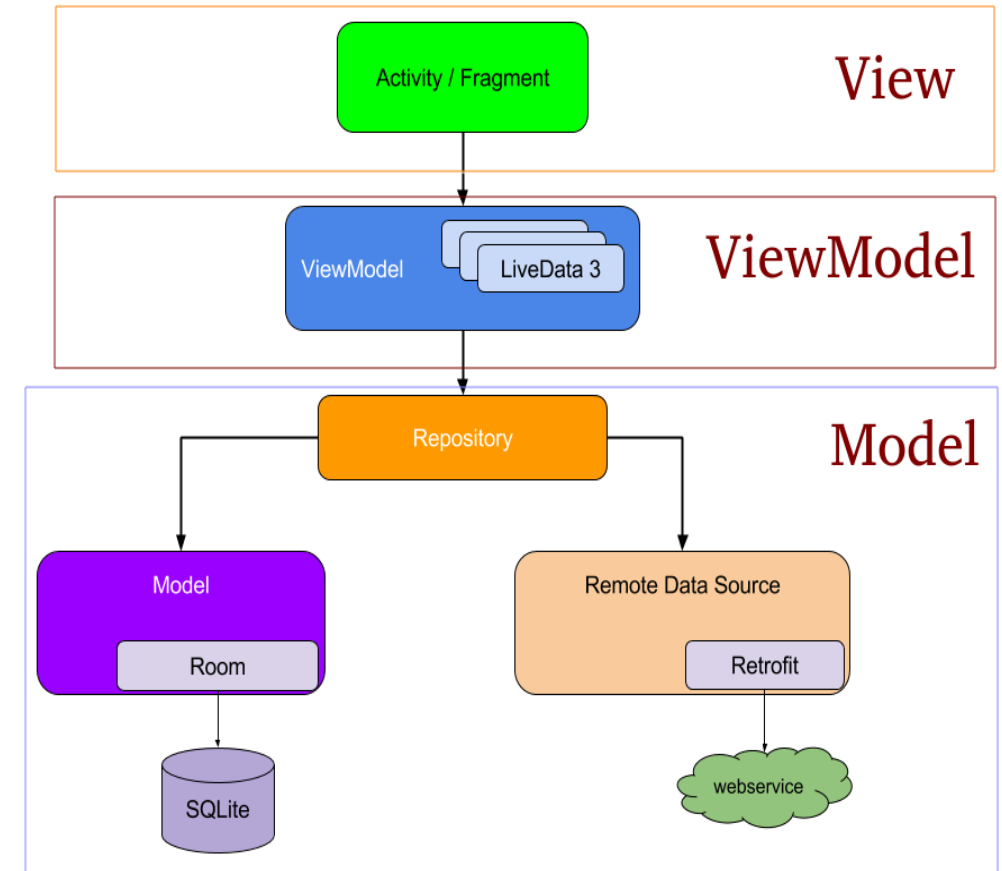
Model keeps ViewModel updated

View Model

OPEN MIND - OPEN DOORS

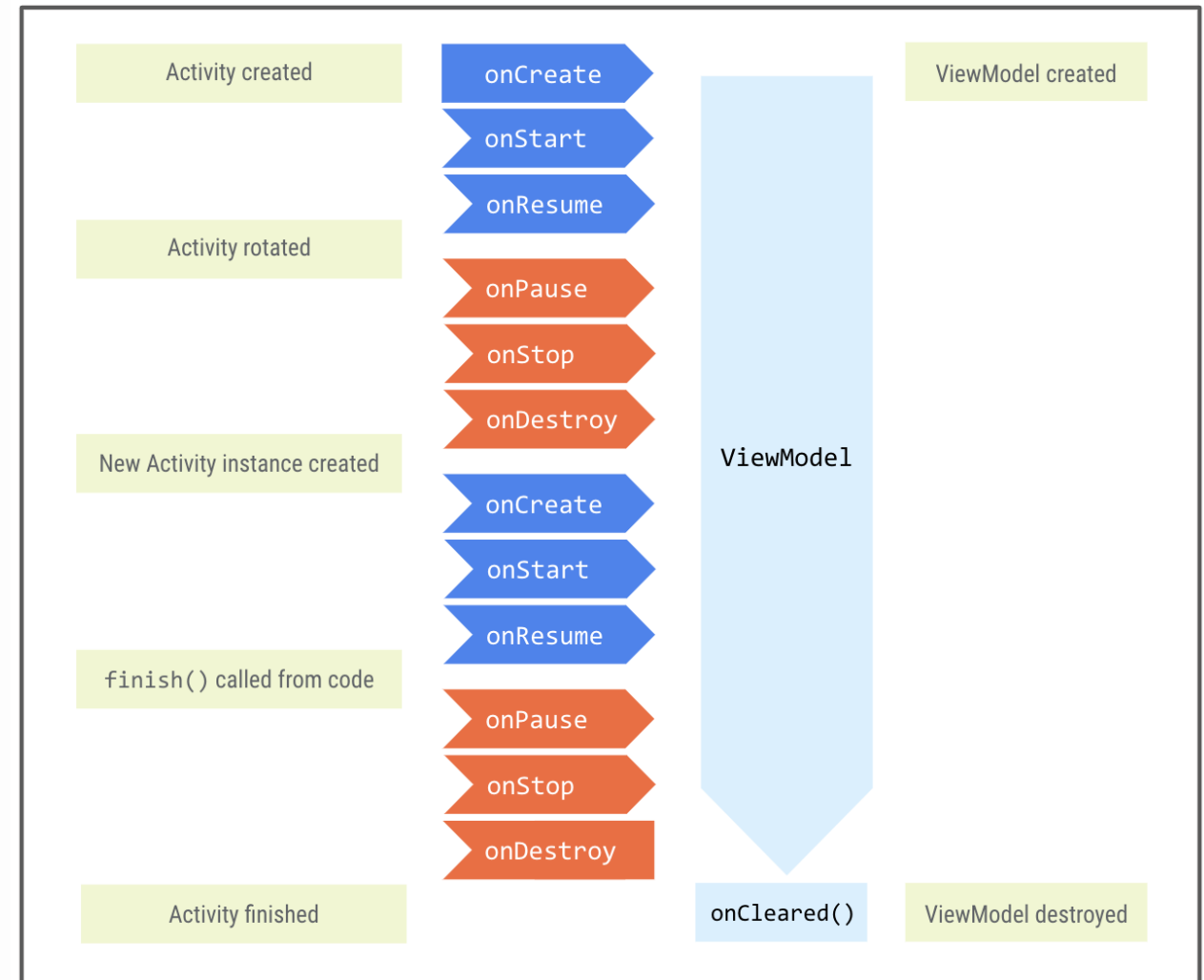
What is ViewModel

- The **ViewModel** class is designed to store and manage UI-related data in a lifecycle conscious way. The ViewModel class allows data to survive configuration changes such as screen rotations
- In this tutorial, we use Viewmodel along with MVVM app architecture. It's recommend architecture design for android app



ViewModel Lifecycles

- The lifecycle of a ViewModel is tied directly to its scope. The ViewModel remains in memory until the Lifecycle it's scoped to goes away permanently:
 - In the case of an activity, when it finishes.
 - In the case of a fragment, when it detaches.
 - In the case of a Navigation entry, when it's removed from the back stack.
- You usually request a ViewModel the first time the system calls an activity object's onCreate() method. The system may call onCreate() several times throughout the life of an activity, such as when a device screen is rotated. The ViewModel exists from when you first request a ViewModel until the activity is finished and destroyed.



Implement a Viewmodel

- Architecture Components provides **ViewModel** helper class for the UI controller that is responsible for preparing data for the UI. ViewModel objects are automatically retained during configuration changes so that data they hold is immediately available to the next activity or fragment instance.
- For example, if you need to display a list of users in your app, make sure to assign responsibility to acquire and keep the list of users to a ViewModel, instead of an activity or fragment, as illustrated by the following sample code:

```
class MyViewModel : ViewModel() {
    private val users: MutableLiveData<List<User>> by lazy {
        MutableLiveData<List<User>>().also {
            loadUsers()
        }
    }

    fun getUsers(): LiveData<List<User>> {
        return users
    }

    private fun loadUsers() {
        // Do an asynchronous operation to fetch users.
    }
}
```

Use a ViewModel

1. Use "by activityViewModels()" or by viewModels() method of fragment-ktx

```
// Use the 'by activityViewModels()' Kotlin property delegate
// from the fragment-ktx artifact
private val model: SharedViewModel by activityViewModels()

override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
    super.onViewCreated(view, savedInstanceState)
    itemSelector.setOnClickListener { item ->
        // Update the UI
    }
}

class MyActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        // Create a ViewModel the first time the system calls an activity's onCreate() method.
        // Re-created activities receive the same MyViewModel instance created by the first activity

        // Use the 'by viewModels()' Kotlin property delegate
        // from the activity-ktx artifact
        val model: MyViewModel by viewModels()
        model.getUsers().observe(this, Observer<List<User>>{ users ->
            // update UI
        })
    }
}
```

Note: to use this ways, you need to add fragment-ktx to build.gradle
implementation "androidx.fragment:fragment-ktx:1.3.6"

Use a ViewModel

2. Use **ViewModelProvider** to get instance of **ViewModel**.

```
class MyViewModel : ViewModel() {  
    private val users: MutableLiveData<List<User>> by lazy {  
        MutableLiveData<List<User>>().also { it: MutableLiveData<List<User>> }  
        loadUsers()  
    }  
  
    fun getUsers(): LiveData<List<User>> {  
        return users  
    }  
  
    private fun loadUsers() {  
        // Do an asynchronous operation to fetch users.  
    }  
}
```

```
class MyActivity : AppCompatActivity() {  
  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContentView(R.layout.activity_main)  
  
        val model: MyViewModel =  
            ViewModelProvider(owner: this).get(MyViewModel::class.java)  
        model.getUsers().observe(owner: this, Observer<List<User>> { users ->  
            // update UI  
        })  
    }  
}
```

Use a ViewModel

2. Use **ViewModelProvider** to get instance of **ViewModel**. But if your ViewModel has parameters in constructor, then you need to define a **ViewModelProvider.Factory** then pass an instance of it as second parameters of **ViewModelProvider**

```
class MyViewModel(private val application: Application) : ViewModel() {
    init {
        //Use application to do something, ex: create room database
    }

    private val users: MutableLiveData<List<User>> by lazy {...}

    fun getUsers(): LiveData<List<User>> {...}

    private fun loadUsers() {...}
}
```

```
class MyViewModelFactory(private val application: Application)
    : ViewModelProvider.Factory {
    override fun <T : ViewModel?> create(modelClass: Class<T>): T {
        if(modelClass.isAssignableFrom(MyViewModel::class.java)){
            return MyViewModel(application = application) as T
        }
        throw IllegalArgumentException()
    }
}
```

```
class MyActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val model: MyViewModel =
            ViewModelProvider( owner: this, MyViewModelFactory(application))
                .get(MyViewModel::class.java)
        model.getUsers().observe( owner: this, Observer<List<User>> { users ->
            // update UI
        })
    }
}
```

Sharing data between Fragments & Activities

- It's very common that two or more fragments in an activity need to communicate with each other or some time you also want to share data between activity to fragments
- Imagine a common case of split-view (master-detail) fragments, where you have a fragment in which the user selects an item from a list and another fragment that displays the contents of the selected item. This case is never trivial as both fragments need to define some interface description, and the owner activity must bind the two together. In addition, both fragments must handle the scenario where the other fragment is not yet created or visible.
- This common pain point can be addressed by using ViewModel objects. These fragments can share a ViewModel using their activity scope to handle this communication, as illustrated by the following sample code

Sharing data	No Sharing Data
by activityViewModels ()	by viewModels()
ViewModelProvider(requireActivity()).get()	ViewModelProvider(this).get()

OPEN MIND - OPEN DOORS

Sharing data example

- In this example, when selected item changed, new data will be observed in both of MasterFragment and DetailFramgent
- If you using **"by viewModels()"** or **"ViewModelProvider(this).get()"** then data will not be share.

```
class SharedViewModel : ViewModel() {  
    val selected = MutableLiveData<Item>()  
  
    fun select(item: Item) {  
        selected.value = item  
    }  
}
```

```
class MasterFragment : Fragment() {  
  
    private lateinit var itemSelector: Selector  
  
    // Use the 'by activityViewModels()' Kotlin property delegate  
    // from the fragment-ktx artifact  
    private val model: SharedViewModel by activityViewModels()  
  
    override fun onCreateView(view: View, savedInstanceState: Bundle?) {  
        super.onCreateView(view, savedInstanceState)  
        itemSelector.setOnClickListener { item ->  
            // Update the UI  
        }  
    }  
}
```

```
class DetailFragment : Fragment() {  
  
    // Use the 'by activityViewModels()' Kotlin property delegate  
    // from the fragment-ktx artifact  
    private val model: SharedViewModel by activityViewModels()  
  
    override fun onCreateView(view: View, savedInstanceState: Bundle?) {  
        super.onCreateView(view, savedInstanceState)  
        model.selected.observe(viewLifecycleOwner, Observer<Item> { item ->  
            // Update the UI  
        })  
    }  
}
```

DataBinding

OPEN MIND - OPEN DOORS

DataBinding

- In Android development, we are using the **findViewById** function to obtain references to views in **runtime**, that is a very inefficient and resource consuming process.
- Data binding will **generates a corresponding binding class** for each layout **at compile time**.
- Advantages of Data binding:
 - ✓ Eliminate findViewById()
 - ✓ Update the values automatically.(doesn't have to keep track of all the ways a value can be updated)
 - ✓ Very effective for UI Testing.
 - ✓ More readable code.
 - ✓ More maintainable code.
 - ✓ Faster development times.
 - ✓ Faster execution times.
 - ✓ Errors can be found during the compile time.
 - ✓ Well suited for MVVM and MVI architectures

How to use DataBinding?

To use data binding in an Android app, we need to do 3 things:

1. Enable databinding in build.gradle file(app level)
2. Add layout tags as outermost tags of the xml file
3. Construct data binding object

Note: For listening LiveData in Fragment, we need set lifecycle owner for binding object, see picture 3

```

7  android {
8      compileSdkVersion 30
9      defaultConfig {...}
10     buildTypes {...}
24     compileOptions {...}
28     kotlinOptions {jvmTarget = '1.8'}
31     buildFeatures {
32         viewBinding true
33         dataBinding true
34     }
35 }
  
```

```

1  <?xml version="1.0" encoding="utf-8"?>
2  <layout xmlns:android="http://schemas.android.com/apk/res/android">
3      <data>
4          <variable
5              name="vm"
6              type="com.ddona.jetpack.viewmodel.CountingViewModel" />
7      </data>
8      <LinearLayout
9          android:layout_width="match_parent"
10         android:layout_height="match_parent"
11         android:orientation="vertical">
12          <TextView
13              android:id="@+id/tvCount"
14              android:layout_width="match_parent"
15              android:layout_height="wrap_content"
16              android:gravity="center_horizontal"
17              android:text="@{vm.count}"
18              android:textSize="22sp" />
19          <Button
20              android:onClick="@{()->vm.increaseValue()}"
21              android:id="@+id/btnCount"
22              android:layout_width="match_parent"
  
```

```

12
13 class CountingOneFragment : Fragment() {
14     private lateinit var binding: FragmentDataBinding
15     private val countingViewModel: CountingViewModel by activityViewModels() {
16         CountingViewModelFactory(application = requireActivity().application)
17     }
18
19     override fun onCreateView(
20         inflater: LayoutInflater,
21         container: ViewGroup?,
22         savedInstanceState: Bundle?
23     ): View {
24         binding = FragmentDataBinding.inflate(inflater, container, attachToRoot: false)
25         binding.lifecycleOwner = viewLifecycleOwner
26         binding.vm = countingViewModel
27
28         binding.btnCount.setOnClickListener { it: View!
29             countingViewModel.increaseValue()
30         }
31         return binding.root
32     }
33 }
  
```

DataBinding – Expression operators

- The expression language looks a lot like expressions found in managed code. You can use the following operators and keywords in the expression language:

- Mathematical `+ - / * %`
- String concatenation `+`
- Logical `&& ||`
- Binary `& | ^`
- Unary `+ - ! ~`
- Shift `>> >>> <<`
- Comparison `== > < >= <=` (Note that `<` needs to be escaped as `<`;))
- `instanceof`

Examples:

```
android:text="@{String.valueOf(index + 1)}"  
android:visibility="@{age > 13 ? View.GONE : View.VISIBLE}"  
android:transitionName="@{"image_" + id}"
```

- Grouping `()`
- Literals - character, String, numeric, `null`
- Cast
- Method calls
- Field access
- Array access `[]`
- Ternary operator `?:`

For more detail, refer [here](#)

DataBinding – Two way binding

- Using one-way data binding, you can set a value on an attribute and set a listener that reacts to a change in that attribute. In below example, the checked will be updated in layout code if you change value in viewmodel

```
<CheckBox
    android:id="@+id/rememberMeCheckBox"
    android:checked="@{viewModel.rememberMe}"
    android:onCheckedChanged="@{viewModel.rememberMeChanged}"
/>
```

- Using two-way data binding, data can be update and listen in both of XML and java/kotlin code. This easy to do by using "@="

```
<CheckBox
    android:id="@+id/rememberMeCheckBox"
    android:checked="@={viewModel.rememberMe}"
/>
```

DataBinding – Two way binding support

- The platform provides built-in support for two-way data binding when you use the attributes in the following table. For detail, please refer: [here](#)

Class	Attribute(s)	Binding adapter
AdapterView	android:selectedItemPosition android:selection	AdapterViewBindingAdapter
CalendarView	android:date	CalendarViewBindingAdapter
CompoundButton	android:checked	CompoundButtonBindingAdapter
DatePicker	android:year android:month android:day	DatePickerBindingAdapter
NumberPicker	android:value	NumberPickerBindingAdapter
RadioButton	android:checkedButton	RadioGroupBindingAdapter
RatingBar	android:rating	RatingBarBindingAdapter
SeekBar	android:progress	SeekBarBindingAdapter
TabHost	android:currentTab	TabHostBindingAdapter
TextView	android:text	TextViewBindingAdapter
TimePicker	android:hour android:minute	TimePickerBindingAdapter

DataBinding – Binding Adapter

- Binding adapters are responsible for making the appropriate framework calls to set values. It allows you to specify the method called to set a value, provide your own binding logic and specify the type of the returned object by using adapter.

```
@BindingAdapter( ...value: "set_url")
fun bindImage(img: ImageView, link: String) {
    Glide.with(img).load(link)
        .placeholder(R.drawable.ic_pokemon)
        .error(R.drawable.pokemon)
        .into(img)
}

@RequiresApi(Build.VERSION_CODES.N)
@BindingAdapter( ...value: "text_html")
fun setText(tv: TextView, data: String) {
    tv.text =
        Html.fromHtml(data, Html.FROM_HTML_MODE_COMPACT)
}
```

Using



```
<ImageView
    android:id="@+id/imgAvatar"
    set_url="@{pokemon.imageUrl}"
    android:layout_width="80dp"
    android:layout_height="80dp"
    android:layout_margin="10dp" />

<TextView
    android:id="@+id/tvName"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_centerInParent="true"
    android:layout_marginStart="15dp"
    android:layout_toEndOf="@id/imgAvatar"
    text_html="@{pokemon.name}"
    android:textStyle="bold" />
```

LiveData

OPEN MIND - OPEN DOORS

LiveData

- LiveData is an observable data holder class. Unlike a regular observable, LiveData is lifecycle-aware, meaning it respects the lifecycle of other app components, such as activities, fragments, or services. This awareness ensures LiveData only updates app component observers that are in an active lifecycle state
- LiveData advantages:
 - ✓ **Ensures your UI matches your data state**
 - ✓ **No memory leaks**
 - ✓ **No crashes due to stopped activities**
 - ✓ **No more manual lifecycle handling**
 - ✓ **Always up to date data**
 - ✓ **Proper configuration changes**
 - ✓ **Sharing resources**

For more detail, refer at [here](#)

- To work with LiveData, following below steps:
 1. Create an instance of LiveData to hold a certain type of data. This is usually done within your ViewModel class.
 2. Create an Observer object that defines the onChanged() method, which controls what happens when the LiveData object's held data changes. You usually create an Observer object in a UI controller, such as an activity or fragment.
 3. Attach the Observer object to the LiveData object using the observe() method. The observe() method takes a LifecycleOwner object. This subscribes the Observer object to the LiveData object so that it is notified of changes. You usually attach the Observer object in a UI controller, such as an activity or fragment

```
class CountingViewModel() : ViewModel() {  
    var count = MutableLiveData<Int>()  
    var message = MutableLiveData<String>()  
}
```

```
val countingViewModel: CountingViewModel by viewModels()  
  
countingViewModel.count.observe(viewLifecycleOwner, { it: Int!  
    binding.tvCount.text = it.toString()  
})
```

Live data issue

- Problems: When you need to handle context-dependent operations from viewmodel. Example: you need to show toast or start an activity when data in viewmodel change, you observe a attribute in viewmodel and start an activity, in some case when you back to activity. The viewmodel observe and get latest data, then it start activity again, you will **NEVER** can back to first activity

```
private val _navigateToDetails = MutableLiveData<Boolean>()

val navigateToDetails : LiveData<Boolean>
    get() = _navigateToDetails

fun userClicksOnButton() {
    _navigateToDetails.value = true
}
```

```
<Button
    android:id="@+id/btnNextActivity"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:onClick="@{()->vm.userClicksOnButton()}"
    android:text="NextActivity" />
```

```
override fun onResume() {
    super.onResume()
    countingViewModel.navigateToDetails.observe(owner: this, { it: Boolean!
        if (it) {
            val intent = Intent(activity, TestEventActivity::class.java)
            startActivity(intent)
        }
    })
}
```

Live data issue

- Solution: You must using SingleLiveEvent or EventWrapper to handle one-time event

```
class SingleLiveEvent<T> : MutableLiveData<T>() {
    private val mPending = AtomicBoolean(initialValue: false)

    @MainThread
    override fun observe(owner: LifecycleOwner, observer: Observer<in T>) {...}

    @MainThread
    override fun setValue(t: T?) {...}

    /** Used for cases where T is Void, to make calls cleaner. ...*/
    @MainThread
    fun call() {...}

    companion object {...}
}
```

CountingViewModel.kt

```
18
19     private val _navigateToDetails = SingleLiveEvent<Any>()
20
21     val navigateToDetails : LiveData<Any>
22         get() = _navigateToDetails
23
24     fun userClicksOnButton() {
25         _navigateToDetails.value = true
26     }
```

```
open class Event<out T>(private val content: T) {

    var hasBeenHandled = false
    private set // Allow external read but not write
    /** Returns the content and prevents its use again. ...*/
    fun getContentIfNotHandled(): T? {
        return if (hasBeenHandled) {
            null
        } else {
            hasBeenHandled = true
            content
        }
    }

    /** Returns the content, even if it's already been handled. ...*/
    fun peekContent(): T = content
}
```

For more detail: refer [here](#)

Problem source code: [here](#)

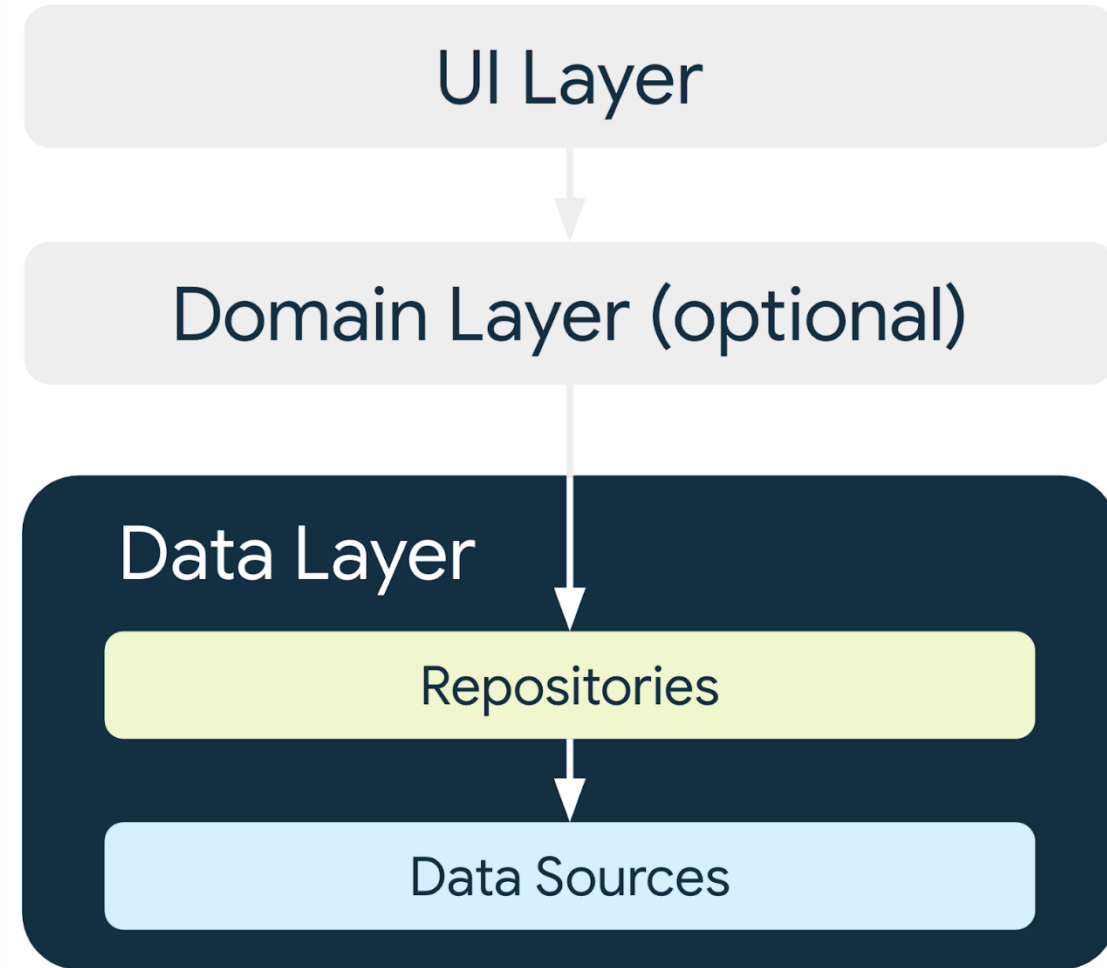
Solution source code: [here](#)

Repository pattern

OPEN MIND - OPEN DOORS

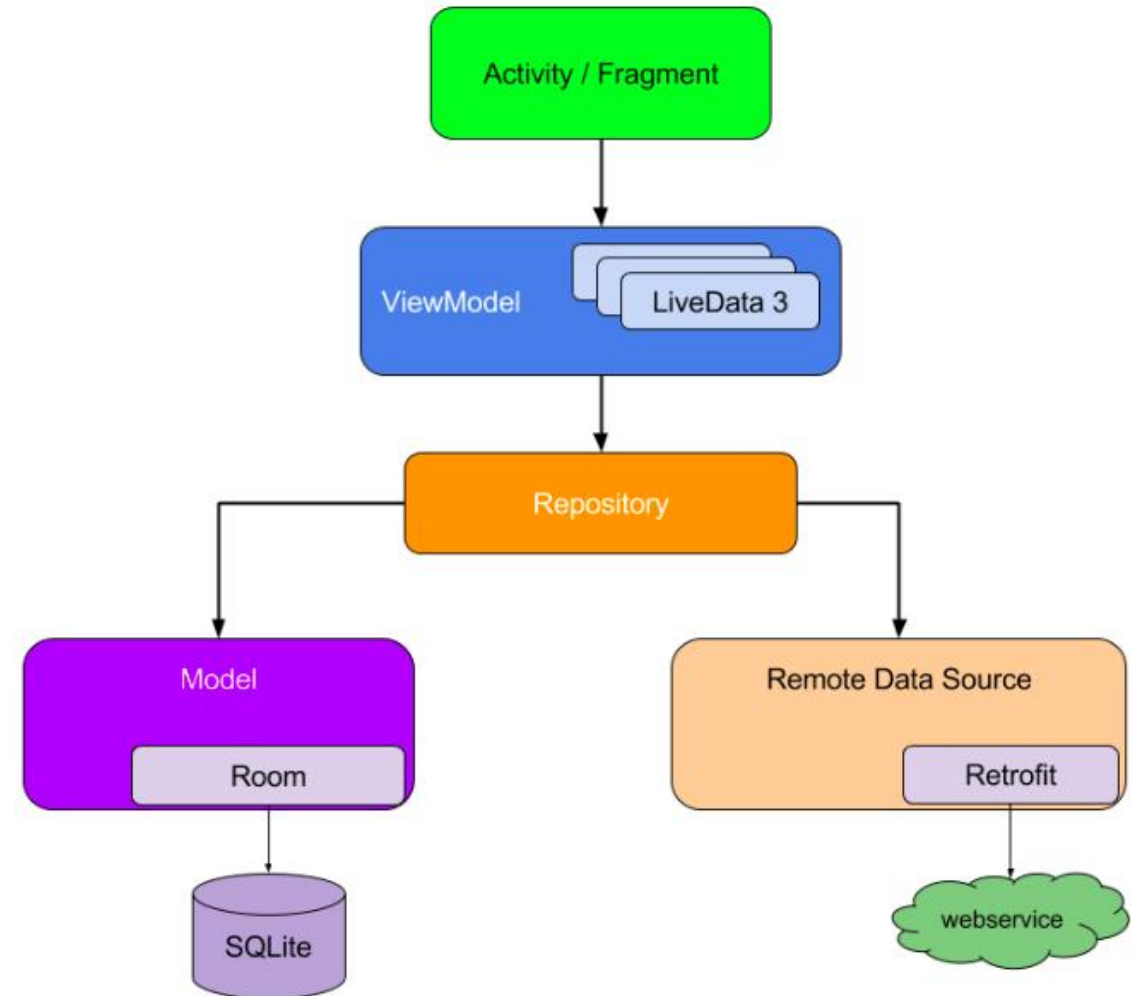
Repository pattern

- The repository pattern is a design pattern that isolates the data layer from the rest of the app. The data layer refers to the part of your app, separate from the UI, that handles the app's data and business logic, exposing consistent APIs for the rest of your app to access this data.
- Repository advantages:
 - Separate processing at Data Layer and Business Layer.
 - Mix Logic perfectly between Local and Remote Data Source before exposing to client.
 - Explicit in communication with Data Source.
 - Independent of Data Source Framework.
 - Optimize performance with specific caching strategy.
 - Reduces duplicate code or missing Logic where the same treatment is applied.
 - Easy to write Unit Test, reduce maintenance risk.
 - Etc...



Repository pattern

- A repository can resolve conflicts between data sources (such as persistent models, web services, and caches) and centralize changes to this data. The diagram below shows how app components such as activities might interact with data sources by way of a repository.
- To implement a repository, you use a separate class. The repository class isolates the data sources from the rest of the app and provides a clean API for data access to the rest of the app. Using a repository class ensures this code is separate from the ViewModel class, and is a recommended best practice for code separation and architecture.





Thank You !

OPEN MIND - OPEN DOORS